

Documentación Técnica

Capture the Flag

Integrantes: - Carlos Alvarez y Samantha Rodas

Contenido

- 1. Introducción
- 2. Objetivos
- 3. Marco Teórico
- 4. Arquitectura del Sistema
- 5. Estructura del Proyecto
- 6. Funcionamiento General del Proyecto
- 7. Explicación de clases
- 8. Evolución del protocolo de comunicación
- 9. Protocolo PRFC v3
- 10. Flujo del juego
- 11. Manual de usuario

1. Introducción

Capture the Flag es un videojuego multijugador desarrollado como proyecto del curso de Redes de Computadoras. El objetivo principal del proyecto no fue únicamente implementar un videojuego funcional, sino diseñar una solución basada en una arquitectura cliente-servidor que permitiera la interoperabilidad entre implementaciones desarrolladas por distintos grupos utilizando diferentes lenguajes de programación.

Para lograr esta interoperabilidad se siguió el protocolo oficial PRFC versión 3, el cual define la estructura de los mensajes intercambiados entre clientes y servidores. Esto garantiza que cualquier cliente que implemente correctamente el protocolo pueda comunicarse con cualquier servidor compatible, independientemente del lenguaje utilizado para su desarrollo.

El servidor mantiene el estado oficial del juego, valida todas las acciones realizadas por los jugadores y sincroniza la información enviada a cada cliente. Por su parte, el cliente únicamente representa gráficamente el estado recibido y envía las acciones del usuario al servidor.

Durante el desarrollo se priorizó la arquitectura del software, la modularidad, el desacoplamiento entre componentes y la organización del código para facilitar su mantenimiento y futuras ampliaciones.

2. Objetivos

Objetivo General

Diseñar e implementar un videojuego multijugador tipo Capture the Flag utilizando una arquitectura cliente-servidor basada en sockets TCP, con descubrimiento inicial mediante UDP y compatible con el protocolo oficial PRFC versión 3.

Objetivos Específicos

- Diseñar una arquitectura modular siguiendo buenas prácticas de ingeniería de software.
- Implementar un servidor autoritativo responsable de toda la lógica del juego.
- Permitir la interoperabilidad con implementaciones desarrolladas en otros lenguajes.
- Utilizar Java Swing para representar el estado del juego.
- Implementar descubrimiento automático del servidor mediante UDP.
- Sincronizar correctamente el estado del juego entre múltiples clientes.
- Facilitar el mantenimiento del proyecto mediante una correcta separación de responsabilidades.

3. Marco Teórico

Arquitectura Cliente-Servidor

Una arquitectura cliente-servidor divide las responsabilidades del sistema en dos partes. El servidor centraliza la lógica y mantiene el estado oficial del sistema, mientras que los clientes solicitan servicios y muestran la información recibida. En este proyecto el servidor valida todos los movimientos y evita inconsistencias entre jugadores.

Sockets

Los sockets constituyen el mecanismo de comunicación utilizado por aplicaciones distribuidas. Permiten el intercambio de datos entre procesos ejecutándose en diferentes equipos mediante protocolos de red como TCP y UDP.

TCP

TCP proporciona una comunicación confiable, orientada a conexión y con entrega ordenada de los datos. Gracias a las confirmaciones (ACK), retransmisiones y control de flujo, resulta adecuado para videojuegos donde la consistencia del estado es más importante que la velocidad absoluta.

UDP

UDP es un protocolo no orientado a conexión y con baja sobrecarga. En este proyecto se utiliza únicamente durante la etapa de descubrimiento del servidor, donde un cliente envía una solicitud de búsqueda y el servidor responde anunciando su disponibilidad.

Bits y Bytes

Toda la información que procesa una computadora se representa mediante bits. Un bit solamente puede tomar dos valores: 0 o 1. Ocho bits forman un byte. Por ejemplo, el valor binario 00000001 representa el número decimal 1, mientras que 11111111 representa el valor 255. La comunicación binaria utilizada por el protocolo PRFC v3 consiste precisamente en enviar secuencias de bytes a través de la red.

Representación Binaria

Los números enteros grandes requieren varios bytes. Por ejemplo, un entero de 32 bits utiliza cuatro bytes consecutivos. Clases como ByteBuffer permiten convertir automáticamente enteros y otros tipos de datos a su representación binaria antes de enviarlos por un socket y reconstruirlos al ser recibidos.

Concurrencia

Como múltiples clientes pueden conectarse simultáneamente, el servidor utiliza hilos de ejecución para atender conexiones concurrentes sin bloquear la ejecución del resto del sistema.

4. Arquitectura del Sistema

El proyecto implementa una arquitectura cliente-servidor. El servidor constituye la autoridad del juego y mantiene el estado oficial de la partida. Los clientes únicamente envían acciones del usuario y representan gráficamente el estado recibido desde el servidor. Esta separación permite mantener la sincronización entre todos los jugadores y evita que un cliente modifique directamente el estado del juego.

Componentes principales

Componente	Responsabilidad
Main	Punto de entrada del sistema.
connect	Administra conexiones TCP/UDP y comunicación.
engine	Contiene la lógica principal del juego.
model	Representa las entidades del dominio.
protocol	Implementa el protocolo PRFC v3.
view	Interfaz gráfica del cliente y servidor.
config	Configuración del sistema.

Flujo general

1. El servidor inicia y carga la configuración.
2. El servidor anuncia su disponibilidad mediante UDP Discovery.
3. El cliente descubre el servidor o se conecta manualmente.
4. Se establece una conexión TCP.
5. Se intercambian mensajes PRFC v3.
6. El servidor actualiza GameSession.
7. El nuevo estado se envía a todos los clientes.
8. Los clientes actualizan la interfaz gráfica.

5. Estructura del Proyecto

Paquete connect

El paquete connect reúne todas las clases encargadas de la comunicación entre el servidor y los clientes. Su responsabilidad principal consiste en establecer conexiones mediante sockets TCP, administrar el intercambio de mensajes y coordinar la comunicación entre los diferentes componentes del sistema.

Dentro de este paquete se encuentran las clases principales Client.java y Server.java, responsables de iniciar las conexiones desde el lado del cliente y del servidor respectivamente. Estas clases administran la creación de sockets, la aceptación de nuevas conexiones, el envío y recepción de mensajes y la comunicación con el resto del sistema.

También se incluye la clase LineConnection, cuya función es abstraer el mecanismo de intercambio de datos entre los extremos de la comunicación, permitiendo que otros componentes trabajen con una interfaz más sencilla sin preocuparse por los detalles de los sockets.

Otra clase importante es GameConfigLoader, encargada de cargar la configuración del servidor desde archivos externos, como puertos, direcciones y otros parámetros necesarios para la inicialización del sistema.

Finalmente, este paquete incorpora distintas clases de registro de eventos (EventLogger), utilizadas para almacenar y mostrar información relacionada con conexiones, errores y eventos importantes ocurridos durante la ejecución del servidor y de los clientes. Esto facilita la depuración del sistema y el seguimiento del funcionamiento interno del proyecto.

Paquete engine

El paquete engine constituye el núcleo funcional del proyecto, ya que concentra toda la lógica relacionada con el desarrollo de la partida.

Su componente más importante es GameSession, considerada el corazón del sistema. Esta clase mantiene el estado actual del juego, recibe las acciones enviadas por los clientes, valida cada movimiento según las reglas establecidas y actualiza continuamente la información de la partida. Además, determina cuándo un jugador captura una bandera, cuándo se produce una condición de victoria y cuándo debe finalizar la partida.

La clase GameInitializer es responsable de preparar el estado inicial del juego antes de comenzar una nueva sesión. Durante este proceso inicializa el mapa, posiciona a los jugadores, crea las banderas y configura los elementos necesarios para que la partida pueda iniciar correctamente.

Por otra parte, GameTickResult representa el resultado producido después de cada actualización del motor del juego. Esta información es utilizada posteriormente para sincronizar el nuevo estado con todos los clientes conectados.

Gracias a esta separación de responsabilidades, la lógica del juego permanece completamente independiente de la interfaz gráfica y del protocolo de comunicación, permitiendo modificar cualquiera de estos componentes sin afectar el funcionamiento interno del motor.

Paquete model

El paquete model contiene todas las entidades que representan el estado interno del videojuego. Estas clases modelan la información utilizada tanto por el servidor como por el motor del juego para mantener sincronizada la partida.

Entre las entidades principales se encuentran Game, Player, Flag, Direction, GameStatus y GameConfig, además de otras clases auxiliares que representan información propia del dominio del problema.

Cada objeto del modelo representa un elemento específico del juego. Por ejemplo, la clase Player almacena la información de cada jugador, incluyendo su posición, estado y demás atributos necesarios para el desarrollo de la partida. De manera similar, la clase Flag representa las banderas que los jugadores deben capturar, mientras que GameStatus mantiene el estado general de la sesión, indicando si la partida se encuentra esperando jugadores, en ejecución o finalizada.

Este paquete no contiene lógica relacionada con la comunicación de red ni con la interfaz gráfica, ya que su propósito es únicamente representar los datos del dominio de forma estructurada y reutilizable.

Paquete protocol

El paquete protocol implementa completamente el protocolo de comunicación PRFC versión 3, responsable del intercambio de información entre clientes y servidor.

Debido a la complejidad del protocolo, este paquete se divide en varios subpaquetes especializados.

El subpaquete core contiene las clases principales relacionadas con la codificación y decodificación de mensajes binarios.

El subpaquete dto almacena los objetos de transferencia de datos (Data Transfer Objects), utilizados para transportar información entre el servidor y los clientes sin exponer directamente las entidades del modelo.

El subpaquete messages define cada uno de los mensajes implementados por el protocolo, representando operaciones como conexión, movimiento, actualización del estado del juego, captura de bandera y finalización de la partida.

El subpaquete enums agrupa todas las enumeraciones utilizadas durante la comunicación, permitiendo identificar de manera consistente tipos de mensajes, estados y diferentes constantes empleadas por el protocolo.

Finalmente, el subpaquete mapping contiene las clases encargadas de convertir las entidades del dominio en mensajes compatibles con el protocolo PRFC v3 y realizar el proceso inverso cuando la información es recibida desde la red.

Esta organización permite mantener el protocolo desacoplado de la lógica del juego, facilitando futuras modificaciones y garantizando la interoperabilidad con implementaciones desarrolladas en otros lenguajes de programación.

Paquete view

El paquete view contiene toda la interfaz gráfica desarrollada utilizando Java Swing. Su responsabilidad consiste exclusivamente en representar visualmente el estado del juego recibido desde el servidor y permitir la interacción del usuario mediante controles gráficos. A diferencia del servidor, este paquete no contiene lógica relacionada con las reglas del juego ni toma decisiones sobre el estado de la partida. Toda la información mostrada en pantalla proviene del servidor, el cual actúa como única autoridad del sistema.

Entre sus principales funciones se encuentran la representación del mapa, la visualización de los jugadores, la actualización de la interfaz durante la partida, la presentación de menús y ventanas de configuración, así como la captura de las acciones realizadas por el usuario para posteriormente enviarlas al servidor mediante el protocolo de comunicación.

La separación entre la interfaz gráfica y la lógica del juego facilita el mantenimiento del proyecto y permite modificar el aspecto visual sin afectar el funcionamiento interno del sistema.

6. Funcionamiento General del Proyecto

Una vez conocida la organización del proyecto y la responsabilidad de cada paquete, es importante comprender cómo interactúan entre sí durante la ejecución del videojuego.

El proyecto sigue una arquitectura cliente-servidor, donde el servidor actúa como la autoridad principal del juego. Esto significa que todas las decisiones relacionadas con el estado de la partida son tomadas por el servidor, mientras que los clientes únicamente envían las acciones realizadas por los jugadores y muestran en pantalla la información recibida.

El flujo general del sistema puede dividirse en varias etapas: inicialización del servidor, descubrimiento del servidor por parte de los clientes, establecimiento de la conexión, inicio de la partida, ejecución del juego y finalización.

Inicialización del Servidor

La ejecución del proyecto comienza cuando el servidor es iniciado. En este proceso se cargan los parámetros de configuración necesarios para el funcionamiento del sistema, tales como los puertos de comunicación, el mapa del juego y demás opciones definidas en los archivos de configuración.

Posteriormente se crea el socket TCP encargado de aceptar las conexiones de los clientes y, paralelamente, se habilita el servicio de descubrimiento mediante UDP para permitir que los jugadores encuentren automáticamente el servidor disponible dentro de la red local.

Una vez completada esta etapa, el servidor permanece en estado de espera hasta que comienzan a conectarse los jugadores.

Descubrimiento del Servidor mediante UDP

Para evitar que el usuario tenga que escribir manualmente la dirección IP del servidor, el proyecto implementa un mecanismo de descubrimiento automático utilizando el protocolo UDP.

Cuando un cliente inicia la búsqueda de partidas disponibles, envía un mensaje de difusión (*broadcast*) a través de la red. Todos los equipos reciben dicho mensaje, pero únicamente el servidor responde indicando su dirección IP, el puerto disponible y la información necesaria para establecer posteriormente la conexión principal.

Debido a que el protocolo UDP no requiere establecer una conexión previa y genera una sobrecarga mínima, resulta adecuado para este proceso de descubrimiento rápido.

Una vez localizado el servidor, el cliente ya conoce la dirección a la que deberá conectarse utilizando TCP.

Establecimiento de la Conexión TCP

Después del descubrimiento, el cliente inicia una conexión TCP con el servidor.

Durante esta fase se realiza el intercambio inicial de información utilizando el protocolo PRFC versión 3, permitiendo registrar al jugador dentro de la sesión de juego y verificar que ambos extremos utilizan el mismo protocolo de comunicación.

A partir de este momento toda la comunicación importante entre cliente y servidor se realiza mediante TCP, garantizando que los mensajes lleguen completos, en el mismo orden en que fueron enviados y sin pérdidas de información.

Este comportamiento resulta indispensable para mantener sincronizado el estado de la partida entre todos los jugadores.

Inicio de la Partida

Una vez que el servidor ha recibido el número suficiente de jugadores, crea una nueva sesión utilizando el motor del juego.

Durante esta etapa se ejecutan procesos como:

- Creación del mapa.
- Inicialización de los jugadores.
- Ubicación de las banderas.
- Configuración del estado inicial.
- Preparación de la sesión de juego.

Cuando todos estos elementos han sido preparados correctamente, el servidor notifica a cada cliente que la partida ha comenzado.

Desde ese instante todos los clientes muestran el mismo estado inicial del juego.

Ejecución de la Partida

Durante el desarrollo de la partida se produce un intercambio continuo de información entre clientes y servidor.

Cada vez que un jugador realiza una acción, como desplazarse por el mapa o intentar capturar una bandera, el cliente envía un mensaje al servidor utilizando el protocolo PRFC v3.

El servidor recibe dicho mensaje y lo entrega al motor del juego (GameSession), donde se valida la acción recibida.

Si la acción cumple las reglas del juego, el estado interno es actualizado y posteriormente el servidor genera nuevos mensajes con la información actualizada para todos los clientes conectados.

De esta forma todos los jugadores observan exactamente el mismo estado del juego.

El siguiente diagrama resume este proceso.



Este ciclo se repite continuamente hasta que ocurre una condición de victoria o finaliza la sesión.

Sincronización del Estado del Juego

El servidor mantiene una única copia válida del estado del juego.

Los clientes no modifican directamente la información de la partida; únicamente muestran los datos enviados por el servidor.

Gracias a este modelo se evita que dos jugadores posean versiones diferentes del mismo juego, reduciendo inconsistencias y evitando posibles problemas derivados de modificaciones locales del estado.

Este enfoque corresponde a un modelo de servidor autoritativo, ampliamente utilizado en videojuegos multijugador debido a su confiabilidad y facilidad para mantener la sincronización entre todos los participantes.

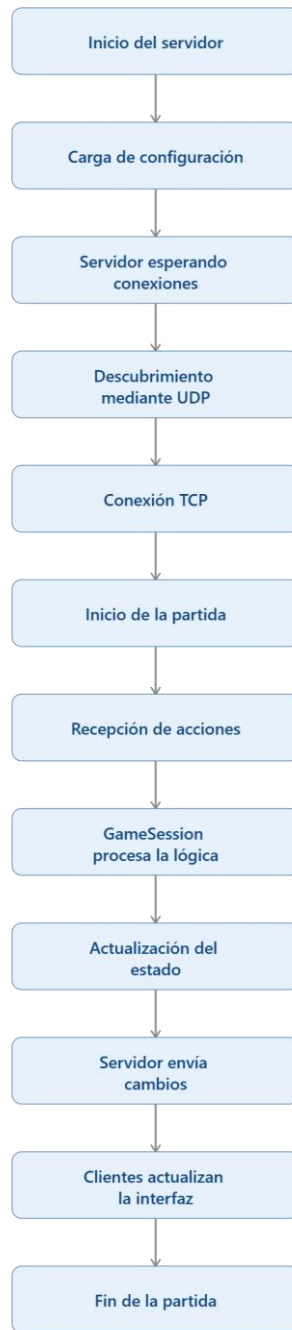
Finalización de la Partida

Cuando el motor del juego detecta una condición de victoria o cualquier otra situación que determine el final de la sesión, actualiza el estado interno de la partida.

Posteriormente el servidor genera un mensaje de finalización utilizando el protocolo PRFC v3 y lo envía a todos los clientes conectados.

Finalmente, la interfaz gráfica presenta el resultado de la partida y el servidor libera los recursos utilizados por la sesión, quedando preparado para iniciar una nueva partida cuando sea necesario.

Resumen del Flujo General



De forma resumida, el funcionamiento completo del sistema puede representarse mediante el siguiente flujo:

7. Explicación de las Clases Principales

En este capítulo se describen las clases más importantes del proyecto y la función que desempeñan dentro del sistema. Debido a la arquitectura modular implementada, cada clase posee una responsabilidad específica, lo que facilita el mantenimiento del código y evita el acoplamiento entre componentes.

Las clases se presentan siguiendo el mismo orden en que participan durante la ejecución del videojuego.

Clase Main

La clase Main constituye el punto de entrada de toda la aplicación.

Su responsabilidad principal consiste en iniciar el programa y permitir que el usuario seleccione el modo de ejecución correspondiente, ya sea iniciar una nueva instancia del servidor o ejecutar un cliente para unirse a una partida existente.

Una vez realizada la selección, la clase crea los objetos necesarios y delega el control al componente correspondiente, evitando concentrar lógica de negocio dentro del método principal.

Gracias a esta implementación, la clase mantiene una única responsabilidad: iniciar correctamente la aplicación.

Responsabilidades principales

- Punto de entrada del programa.
- Inicialización de la aplicación.
- Selección del modo servidor o cliente.
- Creación de los componentes principales.

Clase Server

La clase Server representa el núcleo de la comunicación desde el lado del servidor.

Su función consiste en crear el servidor TCP, aceptar nuevas conexiones y administrar todos los clientes conectados.

Cuando un jugador establece una conexión, esta clase registra al nuevo cliente y coordina el intercambio inicial de mensajes utilizando el protocolo PRFC versión 3.

Además de gestionar las conexiones, mantiene la comunicación con el motor del juego (GameSession), enviando las acciones recibidas por los clientes y distribuyendo posteriormente las actualizaciones generadas por el servidor.

Al tratarse de un servidor autoritativo, todas las decisiones importantes del juego son tomadas desde esta clase en conjunto con el motor del juego.

Responsabilidades principales

- Crear el socket TCP.
- Esperar conexiones entrantes.
- Registrar jugadores.
- Recibir mensajes PRFC v3.
- Enviar actualizaciones a todos los clientes.
- Coordinar la comunicación con GameSession.

Clase Client

La clase Client implementa el comportamiento del cliente del videojuego.

Su responsabilidad consiste en establecer la conexión con el servidor, enviar las acciones realizadas por el jugador y recibir continuamente las actualizaciones del estado del juego.

El cliente no contiene reglas del juego ni modifica directamente el estado de la partida. Su función se limita a transmitir las acciones del usuario y mostrar la información enviada por el servidor.

Este diseño garantiza que todos los clientes mantengan una representación sincronizada del mismo estado del juego.

Responsabilidades principales

- Conectarse al servidor.
- Enviar acciones del jugador.
- Recibir mensajes del servidor.
- Actualizar la interfaz gráfica.
- Mantener la sincronización con la partida.

Clase LineConnection

La clase LineConnection abstrae el proceso de comunicación entre cliente y servidor.

Su objetivo es encapsular las operaciones de envío y recepción de información para que el resto del sistema no tenga que interactuar directamente con los sockets.

Gracias a esta abstracción, tanto el cliente como el servidor pueden intercambiar mensajes utilizando una interfaz mucho más sencilla y reutilizable.

Esta clase contribuye a reducir el acoplamiento entre los componentes encargados de la comunicación y el resto del sistema.

Responsabilidades principales

- Encapsular la comunicación.
- Enviar mensajes.
- Recibir mensajes.
- Ocultar los detalles de los sockets.

Clase GameConfigLoader

La clase GameConfigLoader es responsable de cargar la configuración inicial del sistema.

Durante el inicio del servidor lee los parámetros definidos en el archivo de configuración, como puertos de comunicación y demás propiedades necesarias para el funcionamiento del juego.

Centralizar esta funcionalidad permite modificar la configuración sin necesidad de realizar cambios en el código fuente.

Responsabilidades principales

- Leer archivos de configuración.
- Validar parámetros.
- Crear objetos de configuración.
- Proporcionar la configuración al servidor.

Clase GameSession

La clase GameSession constituye el componente más importante del proyecto y puede considerarse el corazón del videojuego.

En ella reside toda la lógica relacionada con el desarrollo de la partida.

Cada acción enviada por un cliente es recibida por el servidor y posteriormente procesada por esta clase, la cual, valida la solicitud, verifica que cumpla las reglas del juego y actualiza el estado interno cuando corresponde.

Además, controla el movimiento de los jugadores, la captura de banderas, las condiciones de victoria y el estado general de la sesión.

Una vez realizado el procesamiento, genera la información necesaria para que el servidor actualice a todos los clientes.

Responsabilidades principales

- Administrar la partida.
- Procesar acciones de los jugadores.
- Actualizar el estado del juego.
- Validar reglas.
- Detectar condiciones de victoria.
- Generar actualizaciones para los clientes.

Clase GameInitializer

La clase GameInitializer prepara el estado inicial del videojuego antes del comienzo de cada partida.

Durante este proceso crea los objetos necesarios, inicializa el mapa, ubica a los jugadores y posiciona las banderas.

Separar esta responsabilidad del resto del motor facilita el mantenimiento del código y permite reiniciar partidas de forma sencilla.

Clase ProtocolCodec

La clase ProtocolCodec implementa el proceso de serialización y deserialización del protocolo PRFC versión 3.

Su función consiste en convertir los objetos utilizados por el sistema en secuencias de bytes para su transmisión por la red y realizar el proceso inverso cuando los mensajes son recibidos.

Esta clase garantiza que tanto el cliente como el servidor interpreten los mensajes utilizando exactamente el mismo formato.

Responsabilidades principales

- Codificar mensajes.
- Decodificar mensajes.
- Serializar datos.
- Deserializar información.
- Verificar la versión del protocolo.

Clase GameMapper

La clase GameMapper actúa como puente entre el modelo del juego y el protocolo de comunicación.

Su responsabilidad consiste en transformar las entidades del dominio, como jugadores, banderas o estados del juego, en mensajes compatibles con el protocolo PRFC v3.

De igual manera, convierte la información recibida desde la red en objetos que puedan ser procesados por el motor del juego.

Esta separación evita que la lógica del juego dependa directamente de los mensajes enviados por la red.

Clase GameTickResult

La clase representa el resultado obtenido después de cada actualización del motor del juego.

Cada vez que el servidor procesa una acción o ejecuta un ciclo de actualización, esta clase almacena la información necesaria para indicar qué cambios ocurrieron durante ese proceso.

Gracias a esta separación, el motor del juego puede comunicar los resultados obtenidos sin modificar directamente los componentes encargados de la comunicación con los clientes.

Responsabilidades principales

- Almacenar el resultado de una actualización.
- Facilitar la comunicación entre el motor y el servidor.
- Representar los cambios ocurridos durante un ciclo del juego.

Clase Game

La clase Game representa el estado completo de la partida.

En ella se almacenan todos los elementos necesarios para describir el desarrollo del juego, incluyendo los jugadores, las banderas, el estado de la sesión y demás información utilizada por el servidor.

Esta clase funciona como el modelo principal del videojuego y es actualizada constantemente por GameSession.

Responsabilidades principales

- Representar la partida.
- Almacenar jugadores.
- Mantener el estado general.
- Servir como modelo principal del juego.

Clase Player

La clase Player representa a un jugador dentro de la partida.

Cada instancia almacena la información necesaria para identificar al participante y controlar su comportamiento durante el juego, incluyendo su posición, dirección, estado y demás atributos utilizados por el servidor.

El motor del juego utiliza esta información para validar movimientos, detectar colisiones y actualizar el estado de cada jugador.

Responsabilidades principales

- Representar un jugador.
- Mantener su posición.
- Registrar su dirección.
- Identificar al participante.
- Almacenar su estado actual.

Clase Flag

La clase Flag representa las banderas que forman parte del objetivo principal del juego.

Cada bandera posee información sobre su ubicación, su estado y el jugador que la transporta cuando ha sido capturada.

El servidor utiliza esta información para determinar cuándo una bandera ha sido robada, recuperada o entregada correctamente.

Responsabilidades principales

- Representar una bandera.
- Mantener su ubicación.
- Registrar su estado.
- Asociar la bandera con un jugador cuando corresponde.

Clase GameConfig

La clase GameConfig almacena todos los parámetros necesarios para configurar una partida.

Entre estos parámetros pueden encontrarse configuraciones relacionadas con el servidor, límites de jugadores, tiempos de espera y demás valores utilizados durante la inicialización del sistema.

Centralizar esta información facilita modificar la configuración sin alterar la lógica del programa.

Responsabilidades principales

- Almacenar parámetros de configuración.
- Proporcionar la configuración al motor del juego.
- Facilitar la inicialización del servidor.

Clase ProtocolMessage

La clase ProtocolMessage constituye la clase base utilizada por el protocolo PRFC versión 3.

Todos los mensajes intercambiados entre cliente y servidor heredan de esta clase, permitiendo mantener una estructura uniforme durante la comunicación.

Esto simplifica el proceso de codificación y decodificación realizado por el protocolo.

Responsabilidades principales

- Servir como clase base del protocolo.
- Representar un mensaje de red.
- Facilitar el procesamiento uniforme de mensajes.

Clase MessageType

La clase MessageType define todos los tipos de mensajes soportados por el protocolo PRFC versión 3.

Cada mensaje enviado entre cliente y servidor posee un identificador que permite reconocer inmediatamente su propósito.

Gracias a esta enumeración, el protocolo puede determinar rápidamente qué acción debe ejecutarse cuando un mensaje es recibido.

Responsabilidades principales

- Identificar cada tipo de mensaje.
- Facilitar el procesamiento del protocolo.
- Mantener consistencia durante la comunicación.

Clase ProtocolVersion

La clase ProtocolVersion almacena la versión del protocolo utilizada durante la comunicación.

Su objetivo consiste en garantizar que cliente y servidor utilicen exactamente la misma versión del protocolo PRFC, evitando incompatibilidades entre implementaciones diferentes.

Responsabilidades principales

- Representar la versión del protocolo.
- Verificar compatibilidad.
- Evitar errores de comunicación.

Clase ProtocolDecodeException

La clase ProtocolDecodeException representa las excepciones generadas cuando un mensaje recibido no cumple con el formato esperado por el protocolo.

Este mecanismo permite detectar errores durante la decodificación y evita que mensajes inválidos afecten el funcionamiento del servidor.

Responsabilidades principales

- Detectar errores del protocolo.
- Informar fallos durante la decodificación.
- Facilitar el manejo de excepciones.

Clase GameMapper

La clase GameMapper actúa como intermediario entre el modelo del juego y el protocolo PRFC versión 3.

Su función consiste en convertir las entidades del dominio, como jugadores y banderas, en mensajes compatibles con el protocolo y realizar el proceso inverso cuando la información es recibida desde la red.

De esta manera se mantiene desacoplada la lógica del juego del sistema de comunicación.

Responsabilidades principales

- Convertir entidades en mensajes.
- Convertir mensajes en objetos del dominio.
- Mantener separado el protocolo de la lógica del juego.

Clases de la Interfaz Gráfica

El proyecto incorpora varias clases desarrolladas con Java Swing, encargadas de representar visualmente el funcionamiento del sistema.

Entre las más importantes se encuentran:

PanelGame

Representa la ventana principal donde se desarrolla la partida. Muestra el mapa, los jugadores y las banderas, actualizando continuamente la información recibida desde el servidor.

ServerDashboard

Permite visualizar el estado del servidor, las conexiones activas y los eventos registrados durante la ejecución del sistema.

ServerConfigDialog

Proporciona una interfaz gráfica para configurar parámetros del servidor antes de iniciar una nueva sesión.

ServerDiscoveryDialog

Facilita el descubrimiento automático de servidores disponibles mediante el mecanismo de búsqueda implementado con UDP.

8. Evolución del Protocolo de Comunicación

Durante el desarrollo del proyecto, el protocolo de comunicación evolucionó en varias etapas con el objetivo de mejorar la compatibilidad entre las implementaciones, reducir ambigüedades y optimizar el intercambio de información entre clientes y servidor. A continuación, se describen brevemente las principales versiones y las razones que llevaron a adoptar la versión final.

Primera versión: Comunicación mediante JSON

La primera especificación del proyecto definía la comunicación utilizando mensajes en formato JSON codificados en UTF-8 y transmitidos mediante TCP. Cada mensaje era un objeto JSON que incluía información como el tipo de mensaje, la versión del protocolo y los datos necesarios para cada operación del juego.

Este enfoque permitió que los grupos iniciaran el desarrollo de manera sencilla, ya que JSON es un formato ampliamente conocido, fácil de leer y de depurar durante las primeras pruebas.

Desventajas

Conforme el proyecto avanzó, se identificaron varias limitaciones:

- Los mensajes eran considerablemente grandes debido a que cada campo se enviaba como texto.
- Existía un mayor consumo de ancho de banda al transmitir nombres completos de atributos y valores.
- El procesamiento de serialización y deserialización era más costoso.
- La especificación presentaba algunas ambigüedades que podían provocar implementaciones diferentes entre los distintos grupos.

Estas limitaciones motivaron la evolución del protocolo hacia una especificación más eficiente.

Segunda versión (PRFC 2.x)

La segunda versión representó una etapa intermedia en la evolución del protocolo. Aunque todavía utilizaba comunicación basada en texto (JSON), incorporó cambios en la estructura del juego y en la definición de algunos mensajes para mejorar la interoperabilidad entre las distintas implementaciones.

Desventajas

A pesar de las mejoras realizadas, todavía existían aspectos que dificultaban la compatibilidad entre proyectos desarrollados en diferentes lenguajes:

- El uso de JSON seguía generando mensajes de gran tamaño.
- Persistían algunas ambigüedades en la interpretación de determinadas reglas del juego.
- El protocolo continuaba siendo menos eficiente que una solución basada en comunicación binaria.

Estas limitaciones llevaron al diseño de una nueva versión del protocolo.

Versión final: PRFC v3

La versión final del protocolo, PRFC v3, fue la especificación adoptada por todos los proyectos del curso. Esta versión reemplazó la comunicación en JSON por un protocolo binario y definió de forma precisa la estructura de cada mensaje, garantizando que todas las implementaciones fueran compatibles entre sí.

Razones para su adopción

PRFC v3 fue seleccionado como protocolo definitivo porque ofrecía importantes ventajas frente a las versiones anteriores:

- Redujo significativamente el tamaño de los mensajes transmitidos por la red.
- Disminuyó el consumo de ancho de banda y mejoró la velocidad de comunicación.
- Eliminó ambigüedades en la especificación del protocolo.
- Definió un formato binario uniforme para todas las implementaciones.
- Garantizó la interoperabilidad entre clientes y servidores desarrollados en distintos lenguajes de programación.
- Proporcionó una comunicación más eficiente y escalable para el juego multijugador.

A partir de esta versión, el desarrollo del proyecto se realizó utilizando exclusivamente las especificaciones de PRFC v3, las cuales se describen en detalle en el capítulo siguiente.

9. Protocolo PRFC versión 3

Uno de los componentes fundamentales del proyecto es el protocolo de comunicación PRFC versión 3 (Project Remote Frame Communication), encargado de permitir el intercambio de información entre el servidor y todos los clientes conectados.

Este protocolo define la estructura que deben seguir los mensajes transmitidos por la red, asegurando que tanto el cliente como el servidor interpreten la información exactamente de la misma manera.

Gracias al protocolo PRFC v3 es posible enviar acciones del jugador, actualizar el estado de la partida, sincronizar la posición de los participantes, informar eventos importantes y finalizar correctamente una sesión de juego.

El protocolo fue implementado completamente en Java y utiliza comunicación mediante TCP, garantizando que todos los mensajes lleguen completos, en el mismo orden y sin pérdidas de información.

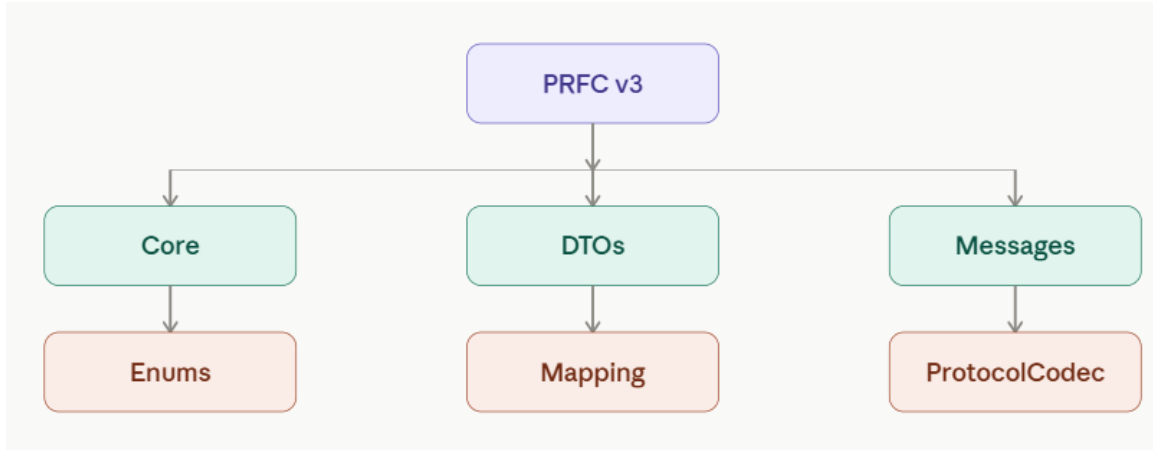
Objetivos del protocolo

El protocolo PRFC versión 3 fue diseñado con los siguientes objetivos:

- Estandarizar la comunicación entre cliente y servidor.
- Mantener sincronizado el estado del juego.
- Reducir la cantidad de información transmitida.
- Facilitar la incorporación de nuevos tipos de mensajes.
- Mantener independencia entre la lógica del juego y la comunicación de red.
- Garantizar compatibilidad entre diferentes versiones del cliente y del servidor.

Arquitectura del protocolo

La implementación del protocolo se encuentra organizada en diferentes componentes especializados.



Cada componente posee una responsabilidad específica que facilita el mantenimiento del protocolo y evita el acoplamiento entre las diferentes capas del sistema.

Componentes principales

Core

Contiene las clases responsables del funcionamiento interno del protocolo, incluyendo la codificación y decodificación de mensajes.

DTO

Los Data Transfer Objects (DTO) transportan únicamente la información necesaria entre cliente y servidor, evitando enviar directamente las entidades del modelo.

Messages

Aquí se encuentran todas las clases que representan los mensajes intercambiados durante la ejecución del juego.

Entre ellos se encuentran:

- JoinRequest
- JoinAccepted
- ChangeDirectionRequest
- InteractRequest
- GameStateMessage
- LobbyStateMessage
- FlagPickedUpMessage
- GameOverMessage

Cada mensaje representa una acción específica del sistema.

Enums

Agrupar las enumeraciones utilizadas por el protocolo.

Por ejemplo:

- MessageType
- Direction
- ErrorCode
- JoinRejectedReason
- GameOverReason

Estas enumeraciones permiten interpretar correctamente cada mensaje recibido.

Mapping

Convierte las entidades del juego en mensajes del protocolo y viceversa, manteniendo separadas la lógica del juego y la comunicación de red.

Representación de la información en formato binario

A diferencia de protocolos basados en texto, como JSON o XML, PRFC versión 3 utiliza comunicación binaria.

Esto significa que las instrucciones no se envían como palabras legibles para una persona, sino como secuencias de bits organizadas en bytes.

Cuando un jugador realiza una acción, el cliente convierte esa información en datos binarios antes de enviarla al servidor.

Este mecanismo ofrece varias ventajas:

- Mayor velocidad de transmisión.
- Menor consumo de ancho de banda.
- Menor tiempo de procesamiento.
- Mensajes de tamaño reducido.
- Mayor eficiencia durante la comunicación.

Estructura general de un mensaje

Todos los mensajes enviados mediante PRFC siguen una estructura previamente definida.

De forma simplificada, un mensaje está compuesto por:

Campo	Tamaño	Descripción
Tipo de mensaje	1 byte	Identifica la operación que se desea realizar.
Longitud	2 bytes	Tamaño de los datos enviados.
Datos	Variable	Información específica del mensaje.

Gracias a esta estructura el receptor puede interpretar correctamente la información recibida.

¿Qué representan los bits?

Cada byte enviado por la red tiene un significado específico.

Por ejemplo, cuando un jugador solicita unirse a una partida, el cliente no envía el texto:

JOIN

En realidad, transmite una secuencia binaria similar a:

00000001

00000000 00001000

01010000 01101100 01100001 01111001 01100101 01110010

La interpretación sería:

Información	Significado
00000001	Tipo de mensaje (JoinRequest)
00000000 00001000	Longitud del mensaje
Resto de bytes	Nombre del jugador

El servidor lee primero el tipo de mensaje y posteriormente interpreta el resto de los datos según la estructura definida por el protocolo.

Ejemplo de una instrucción

Supóngase que un jugador desea desplazarse hacia arriba.

El cliente crea un objeto `ChangeDirectionRequest`, el cual posteriormente es convertido por `ProtocolCodec` en una secuencia binaria.

Conceptualmente podría representarse así:

Campo	Valor
Tipo de mensaje	<code>ChangeDirectionRequest</code>
Dirección	Norte
ID del jugador	5

En formato binario:

00000010

00000001

00000000 00000000 00000000 00000101

Cada grupo de bits representa una parte distinta del mensaje.

El servidor recibe exactamente esos bytes y reconstruye nuevamente el objeto Java correspondiente.

Interpretación de las instrucciones

El primer byte del mensaje indica al servidor qué operación debe ejecutar.

Conceptualmente, el protocolo funciona de la siguiente manera:

Código	Acción
00000001	JoinRequest
00000010	ChangeDirectionRequest
00000011	InteractRequest
00000100	LobbyStateMessage
00000101	GameStartedMessage
00000110	GameStateMessage
00000111	FlagPickedUpMessage
00001000	FlagStolenMessage
00001001	GameOverMessage
00001010	ErrorMessage

Nota: Los códigos mostrados corresponden a una representación conceptual para explicar el funcionamiento del protocolo. En la implementación real, los valores exactos están definidos por la enumeración `MessageType` del proyecto.

Serialización y deserialización

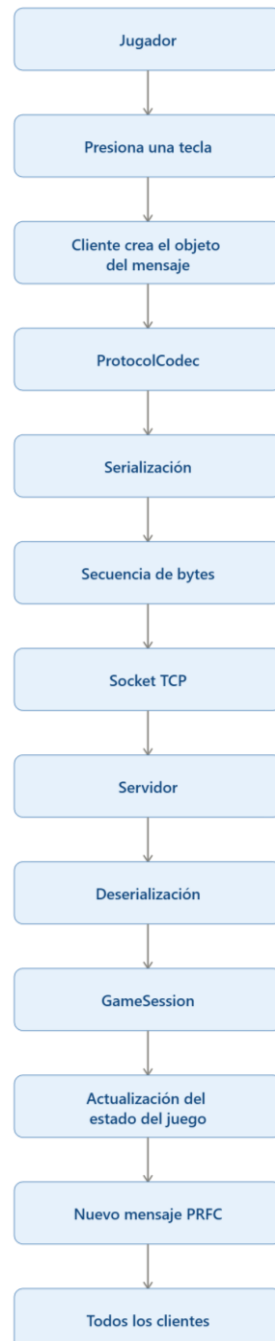
El proceso mediante el cual un objeto Java se convierte en una secuencia de bytes recibe el nombre de serialización.

En el proyecto esta tarea es realizada por la clase ProtocolCodec, la cual organiza la información dentro de un ByteBuffer antes de enviarla mediante el socket TCP.

Cuando el servidor recibe el mensaje ocurre el proceso inverso, denominado deserialización, donde la secuencia de bytes vuelve a convertirse en un objeto Java que posteriormente será procesado por GameSession.

Flujo de una instrucción

El recorrido completo de una acción realizada por el jugador puede resumirse de la siguiente manera:



Flujo de comunicación

Durante toda la partida el protocolo sigue el siguiente ciclo:

1. El jugador realiza una acción.
2. El cliente crea el mensaje correspondiente.
3. El mensaje es serializado mediante ProtocolCodec.
4. Los bytes son enviados por TCP.
5. El servidor recibe la información.
6. El mensaje es deserializado.
7. GameSession procesa la acción.
8. Se actualiza el estado del juego.
9. El servidor genera un nuevo mensaje.
10. Todos los clientes reciben la actualización.

Este ciclo ocurre continuamente mientras la partida permanece activa.

Ventajas del protocolo PRFC versión 3

La implementación del protocolo proporciona múltiples beneficios:

- Comunicación uniforme entre cliente y servidor.
- Mayor velocidad gracias al uso de mensajes binarios.
- Menor consumo de ancho de banda.
- Fácil incorporación de nuevos tipos de mensajes.
- Separación entre la lógica del juego y la comunicación.
- Mayor facilidad para el mantenimiento del código.
- Compatibilidad entre distintas versiones del protocolo.

10. Flujo del Juego

Inicio de la aplicación

Cuando el usuario ejecuta la aplicación, el sistema permite seleccionar si se desea iniciar una nueva instancia del servidor o conectarse como cliente a una partida existente.

Si el usuario inicia el servidor, esta carga la configuración del juego, prepara el mapa, habilita el servicio de descubrimiento mediante UDP y comienza a esperar conexiones de jugadores.

Si el usuario inicia el cliente, puede buscar automáticamente los servidores disponibles mediante el mecanismo de descubrimiento o introducir manualmente la dirección IP del servidor.

Conexión de los jugadores

Una vez localizado el servidor, el cliente establece una conexión TCP utilizando el protocolo PRFC versión 3.

Durante este proceso el servidor registra al nuevo jugador dentro de la sesión y envía la información necesaria para sincronizar el estado inicial del juego.

Todos los jugadores conectados permanecen en la sala de espera hasta que el servidor inicia la partida.

Inicio de la partida

Cuando existe el número necesario de jugadores, el servidor crea una nueva sesión utilizando el motor del juego.

En esta etapa se realizan las siguientes acciones:

- Inicialización del mapa.
- Creación de los jugadores.
- Posicionamiento de las banderas.
- Configuración del estado inicial.
- Sincronización con todos los clientes.

Después de completar estas operaciones, el servidor notifica el inicio de la partida a todos los participantes.

Desarrollo del juego

Durante la partida cada jugador puede desplazarse por el mapa e interactuar con los distintos elementos del escenario.

Cada acción realizada por el usuario es enviada al servidor mediante mensajes PRFC versión 3.

El servidor valida la solicitud utilizando GameSession y, si la acción es válida, actualiza el estado oficial del juego.

Posteriormente genera nuevos mensajes con la información actualizada y los envía a todos los clientes para mantener la sincronización.

Captura de la bandera

El objetivo principal consiste en capturar la bandera y transportarla afuera del radio designado.

Durante este proceso el servidor verifica continuamente que se cumplan las condiciones necesarias para realizar la captura y actualiza el estado de la partida cuando una bandera es tomada, recuperada o entregada.

Todos estos eventos son comunicados inmediatamente a los clientes mediante el protocolo PRFC versión 3.

Finalización de la partida

La partida finaliza cuando se cumple una condición de victoria establecida por las reglas del juego.

El servidor actualiza el estado interno, genera el mensaje correspondiente de finalización y lo envía a todos los clientes conectados.

Finalmente, cada cliente presenta el resultado de la partida y queda preparado para iniciar una nueva sesión.

11. Manual de Usuario

Este manual describe el procedimiento para ejecutar el proyecto Capture the Flag utilizando Git Bash, así como los pasos necesarios para iniciar el servidor, conectar clientes y comenzar una partida.

El proyecto fue desarrollado bajo una arquitectura cliente-servidor, por lo que siempre debe existir una instancia del servidor antes de que los clientes puedan conectarse.

Compilación del Proyecto

Desde Git Bash se compila el proyecto utilizando el script o Makefile proporcionado.

make

Al finalizar este proceso se generarán los archivos compilados dentro del directorio correspondiente.

Inicio del Servidor

Para iniciar una nueva partida se ejecuta el servidor desde Git Bash.

make run-server

Una vez iniciado, el servidor:

- carga la configuración,
- inicializa el mapa,
- habilita el descubrimiento mediante UDP,
- espera conexiones TCP de los clientes.

Inicio del Cliente

Cada jugador abre una nueva ventana de Git Bash y ejecuta:

make run-client

o el comando correspondiente definido en el proyecto.

Después aparecerá la interfaz gráfica del cliente.

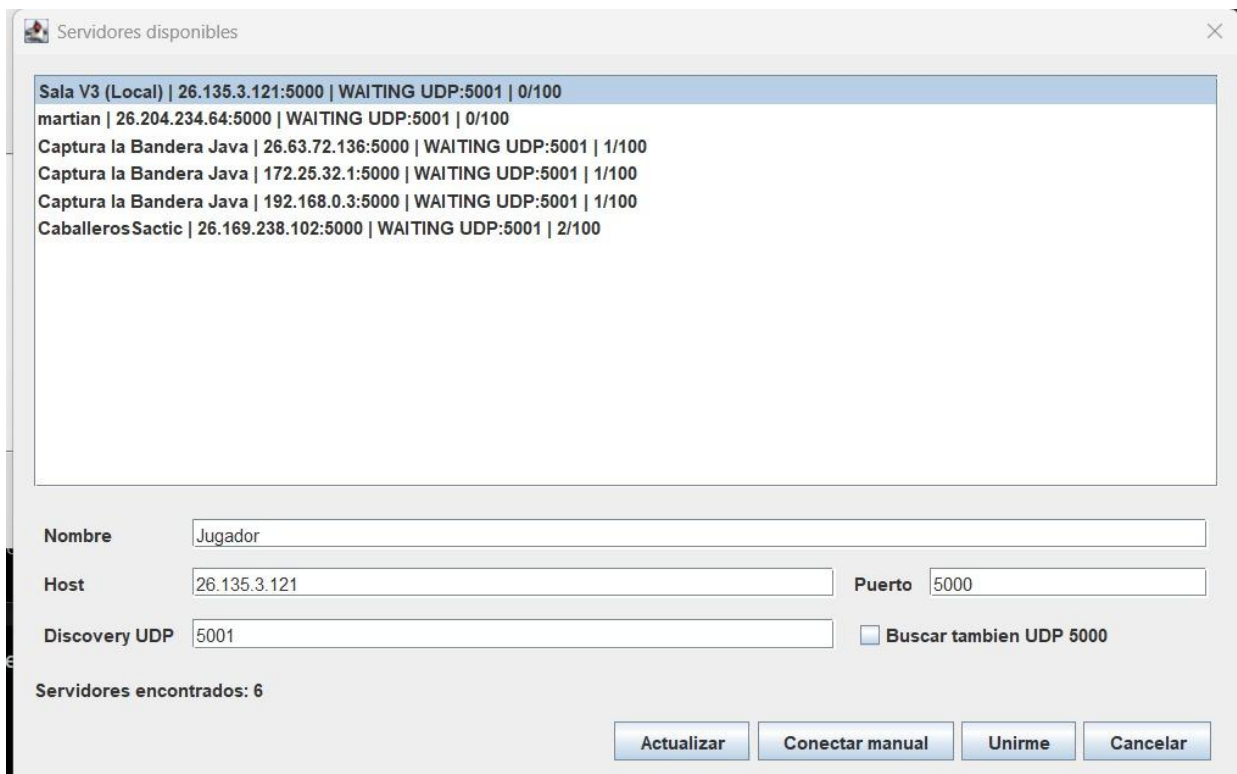
Descubrimiento Automático

Al abrir el cliente se puede seleccionar la opción Actualizar para buscar automáticamente los servidores disponibles.

El cliente envía una solicitud mediante UDP y los servidores activos responden con:

- nombre de la sala,
- dirección IP,
- puerto TCP,
- estado de la partida.

Los servidores encontrados aparecen en la lista para que el usuario pueda seleccionarlos.



The screenshot shows a window titled "Servidores disponibles" with a list of servers and a form for connection details.

Nombre	IP	Estado	Porto	Progreso
Sala V3 (Local)	26.135.3.121	5000	WAITING UDP:5001	0/100
martian	26.204.234.64	5000	WAITING UDP:5001	0/100
Captura la Bandera Java	26.63.72.136	5000	WAITING UDP:5001	1/100
Captura la Bandera Java	172.25.32.1	5000	WAITING UDP:5001	1/100
Captura la Bandera Java	192.168.0.3	5000	WAITING UDP:5001	1/100
CaballerosSactic	26.169.238.102	5000	WAITING UDP:5001	2/100

Form fields:

- Nombre:
- Host:
- Porto:
- Discovery UDP:
- ☐ Buscar tambien UDP 5000

Servidores encontrados: 6

Buttons: Actualizar, Conectar manual, Unirme, Cancelar

Pantalla Cliente

Conexión Manual

Si el servidor no aparece automáticamente, el usuario puede seleccionar Conectar manual e ingresar:

- Dirección IP del servidor.
- Puerto TCP.
- Nombre del jugador.

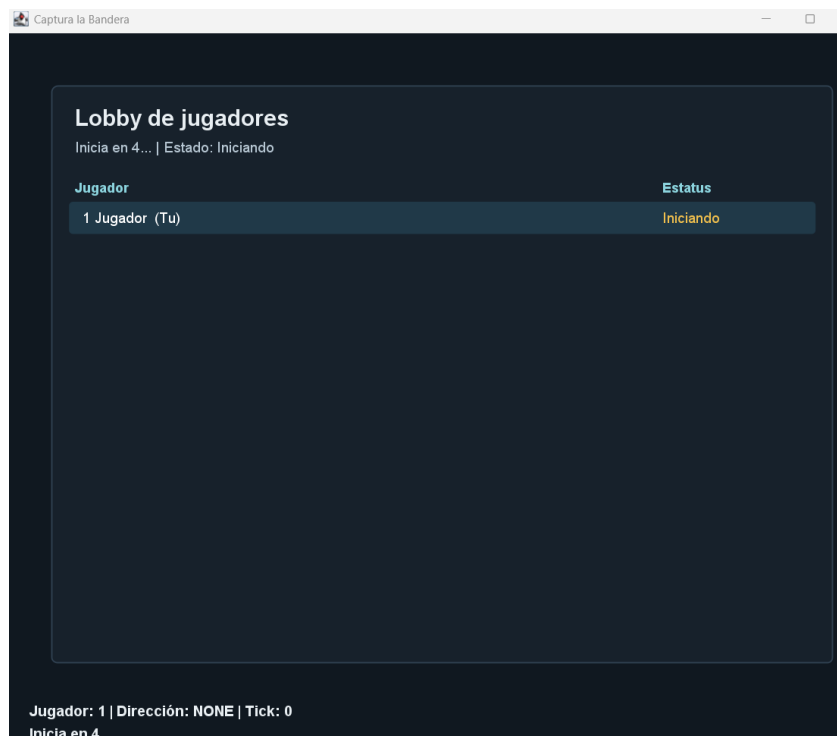
Una vez establecida la conexión, el servidor registra al nuevo participante y lo incorpora a la sala de espera.

Inicio de la Partida

Cuando todos los jugadores se encuentran conectados, el servidor inicia la sesión.

Durante este proceso:

- se inicializa el mapa,
- se posicionan los jugadores,
- se colocan las banderas,
- se sincroniza el estado inicial con todos los clientes.



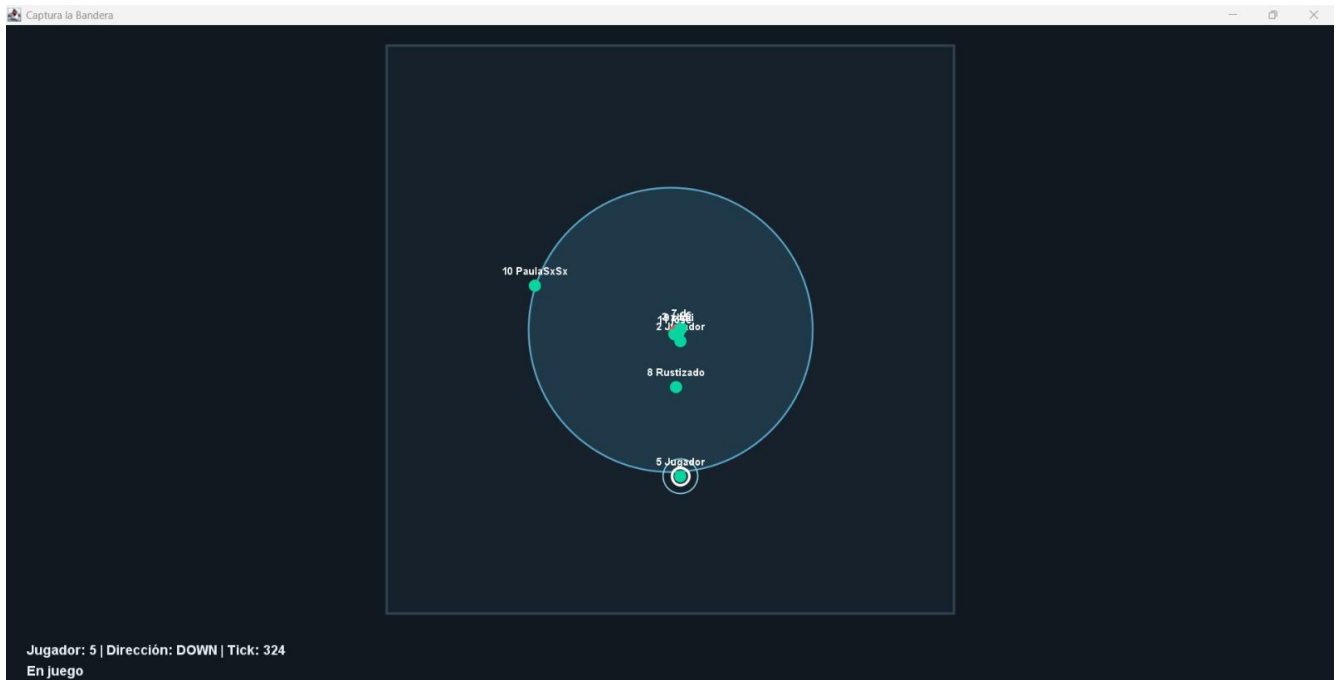
Desarrollo del Juego

Durante la partida el jugador puede:

- desplazarse por el mapa;
- capturar la bandera;
- robar la bandera;
- interactuar con el entorno;
- salir del radio y ganar.

Cada acción es enviada al servidor mediante el protocolo PRFC versión 3.

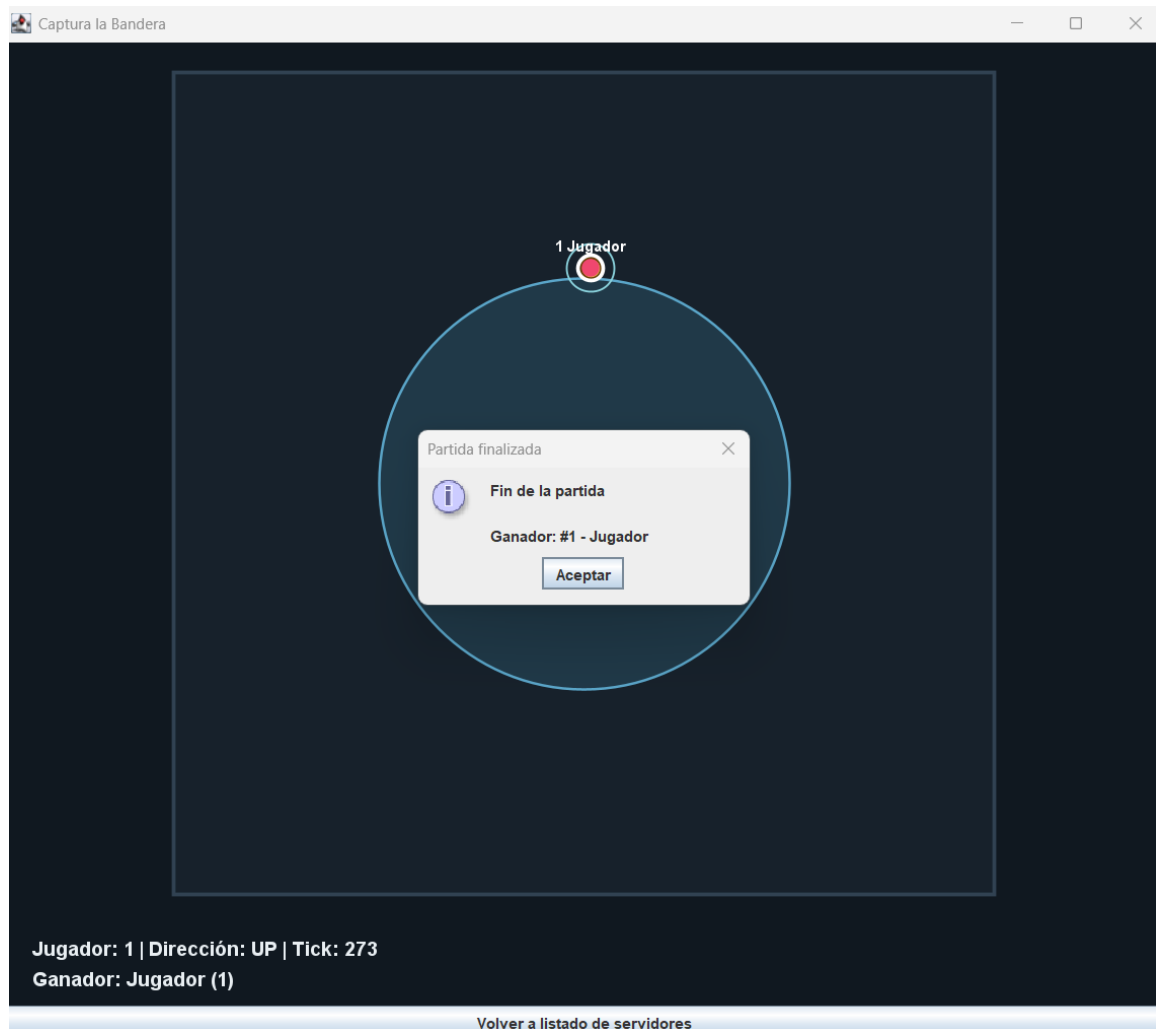
El servidor valida la acción y distribuye el nuevo estado del juego a todos los clientes conectados.



Finalización de la Partida

Cuando se cumple una condición de victoria, el servidor:

- actualiza el estado de la sesión;
- envía el mensaje de finalización;
- informa el equipo ganador;
- deja el sistema preparado para iniciar una nueva partida.



Crear Servidor

Antes de iniciar una nueva partida es necesario crear una instancia del servidor. Para ello, el sistema presenta la ventana Crear partida, donde el administrador configura los parámetros básicos de la sesión. En esta ventana se deben completar los siguientes campos:

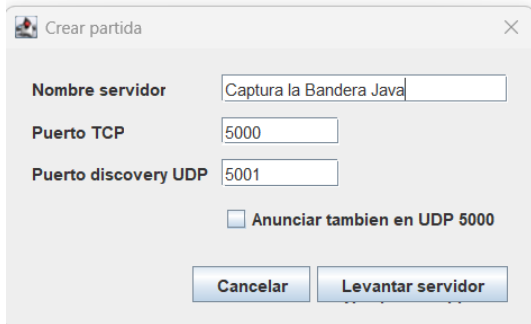
- Nombre del servidor: identifica la partida que será visible para los clientes durante el proceso de descubrimiento automático. Este nombre aparecerá en la lista de servidores disponibles.
- Puerto TCP: corresponde al puerto utilizado para establecer la comunicación principal entre el servidor y los clientes. Una vez que un jugador selecciona una partida, toda la comunicación del juego se realiza a través de este puerto utilizando el protocolo TCP.
- Puerto Discovery UDP: define el puerto empleado por el mecanismo de descubrimiento automático. Cuando un cliente busca partidas disponibles, envía una solicitud mediante UDP y el servidor responde anunciando su disponibilidad utilizando este puerto.
- Anunciar también en UDP 5000: esta opción permite que el servidor responda adicionalmente a solicitudes realizadas sobre el puerto UDP 5000. Su objetivo es proporcionar compatibilidad con implementaciones que utilicen dicho puerto para el descubrimiento de servidores.

Una vez configurados estos parámetros, el administrador debe presionar el botón Levantar servidor para iniciar la sesión.

Al hacerlo, el sistema realiza automáticamente las siguientes acciones:

- Carga la configuración establecida por el usuario.
- Inicializa el servidor TCP.
- Activa el servicio de descubrimiento mediante UDP.
- Prepara el estado inicial del juego.
- Queda a la espera de conexiones de los clientes.

Si el administrador decide no iniciar la partida, puede seleccionar el botón Cancelar, con lo cual la ventana se cerrará sin crear el servidor.



Panel del Servidor

Al iniciar el servidor se muestra la ventana principal de administración, desde la cual es posible supervisar el estado de la partida y controlar la sesión de juego.

La ventana presenta la siguiente información:

- **TCP:** indica el puerto utilizado para las conexiones de los clientes.
- **Discovery UDP:** muestra el puerto utilizado para el descubrimiento automático de servidores.
- **Estado:** informa el estado actual de la partida (WAITING, RUNNING o FINISHED).
- **Jugadores:** muestra la cantidad de jugadores conectados al servidor.
- **Jugadores conectados:** presenta una lista con todos los participantes que se han unido a la sesión.

En la parte inferior también se encuentran los botones para administrar la partida.

Estado del servidor

El servidor puede encontrarse en alguno de los siguientes estados:

- **WAITING:** el servidor se encuentra disponible y esperando que los jugadores se conecten.
- **RUNNING:** la partida se encuentra en desarrollo y los jugadores pueden interactuar normalmente.
- **FINISHED:** la partida ha concluido y el servidor permanece disponible para iniciar una nueva sesión.

Conexión de los Jugadores

Una vez que el servidor está en ejecución, los clientes pueden localizarlo mediante el mecanismo de descubrimiento automático utilizando UDP o conectarse manualmente ingresando la dirección IP y el puerto TCP.

Cada vez que un jugador establece una conexión correctamente, el servidor realiza las siguientes acciones:

- Registra al nuevo jugador dentro de la sesión.
- Actualiza el contador de jugadores conectados.
- Agrega el nombre del jugador a la lista de participantes.
- Sincroniza la información inicial de la partida con el nuevo cliente.

De esta manera, el administrador puede verificar en tiempo real cuántos jugadores se encuentran conectados y quiénes forman parte de la sesión.

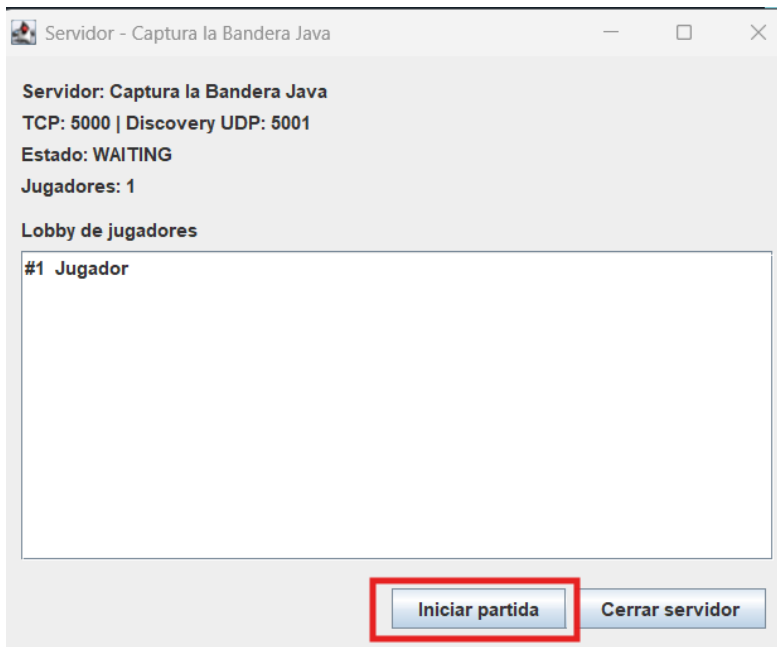
Inicio de la Partida

Cuando todos los jugadores necesarios se han conectado, el servidor inicia la partida.

Durante este proceso el motor del juego realiza automáticamente las siguientes tareas:

- Inicializa el mapa.
- Ubica a los jugadores en sus posiciones iniciales.
- Coloca las banderas correspondientes.
- Configura el estado inicial de la sesión.
- Envía el estado del juego a todos los clientes conectados.

A partir de este momento el estado del servidor cambia de WAITING a RUNNING, indicando que la partida ha comenzado.



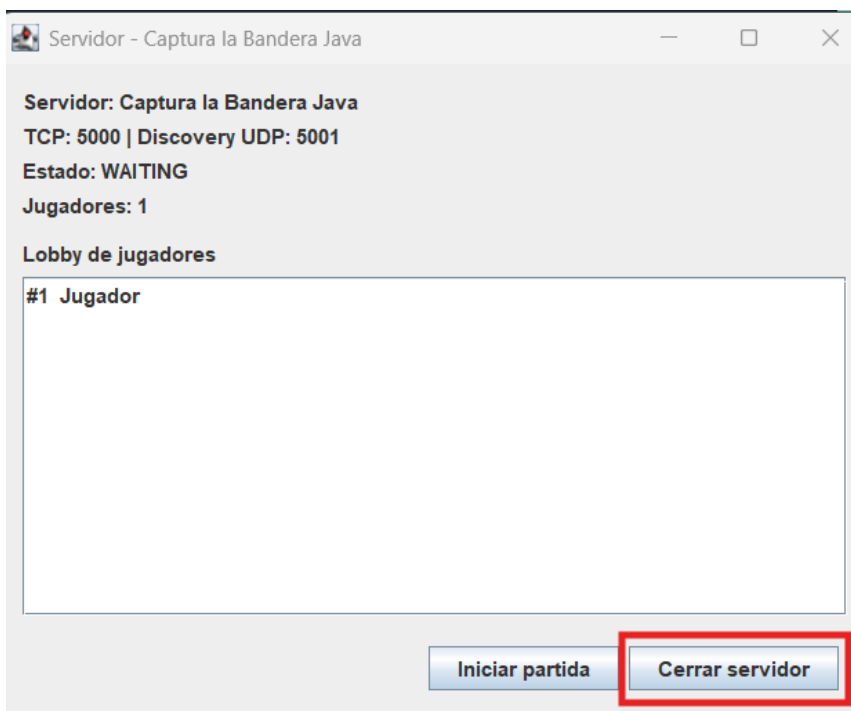
Finalización de la Partida

La partida finaliza cuando se cumple una de las condiciones de victoria establecidas por las reglas del juego.

Al finalizar la sesión, el servidor:

- Actualiza el estado de la partida a FINISHED.
- Envía el mensaje de finalización a todos los clientes mediante el protocolo PRFC versión 3.
- Informa el resultado de la partida.
- Libera los recursos utilizados por la sesión.

Los clientes reciben esta información y muestran el resultado correspondiente en la interfaz gráfica.

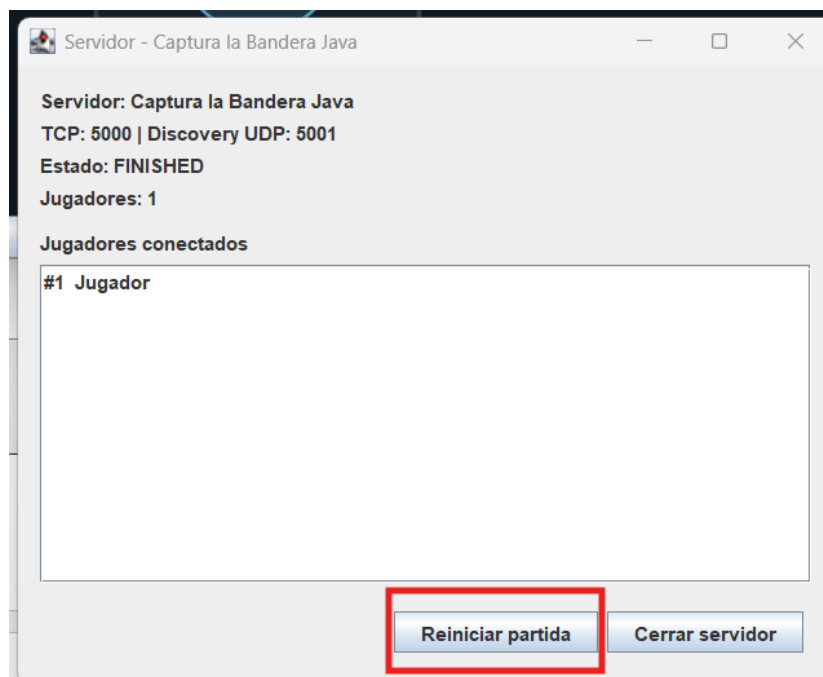


Reiniciar una Partida

Después de finalizar una sesión, el administrador puede utilizar el botón Reiniciar partida para crear una nueva partida sin necesidad de cerrar el servidor.

Al reiniciar la sesión se realizan nuevamente las siguientes acciones:

- Se restablece el estado interno del juego.
- Se reinicializa el mapa.
- Se reposicionan los jugadores.
- Se restauran las banderas a su ubicación inicial.
- Se sincroniza el nuevo estado con todos los clientes conectados.
- Esto permite comenzar una nueva partida de forma rápida manteniendo la misma conexión entre el servidor y los clientes.



12. Implementación y Cronología del Proyecto

Inicio del proyecto

- Análisis del reglamento.
- Selección de Java como lenguaje.
- Definición de la arquitectura Cliente-Servidor.
- Creación del repositorio Git.

Evolución del protocolo

Durante el desarrollo del proyecto el protocolo evolucionó desde una especificación basada en JSON hasta la versión definitiva PRFC v3. La descripción completa de cada versión se presenta en el Capítulo 8.

Así no repites información.

Desarrollo del servidor

- creación del servidor TCP
- descubrimiento UDP
- GameSession
- sincronización

Desarrollo del cliente

- Swing
- captura de teclado
- render
- comunicación TCP

Integración

Una vez desarrollados los componentes principales del proyecto, se inició la etapa de integración entre el cliente y el servidor. El objetivo fue verificar que ambos sistemas pudieran comunicarse correctamente utilizando el protocolo definido para el proyecto y que todas las acciones realizadas por el usuario se reflejaran de forma sincronizada en la partida.

El proceso de integración se realizó de manera gradual. Inicialmente se verificó el establecimiento de la conexión TCP entre cliente y servidor, comprobando el intercambio de los mensajes de conexión (JOIN y JOIN_ACCEPTED). Posteriormente se integró el envío y recepción de los mensajes correspondientes al movimiento de los jugadores, validando que el servidor actualizara el estado del juego y enviara dicha información a todos los clientes conectados.

Una vez establecida la comunicación básica, se incorporaron las funcionalidades restantes del protocolo, como el inicio de la partida, la actualización del estado del juego (GAME_STATE), la captura y robo de la bandera, las desconexiones de jugadores y la finalización de la partida.

Finalmente, se realizaron pruebas de interoperabilidad para comprobar que la implementación pudiera comunicarse con clientes y servidores desarrollados por otros grupos utilizando la misma especificación del protocolo PRFC v3. Durante esta etapa también se corrigieron problemas de compatibilidad relacionados con la comunicación en red, garantizando que todos los mensajes fueran interpretados correctamente por las distintas implementaciones.

13. Control de Versiones (Git)

Durante todo el desarrollo del proyecto se utilizó Git como sistema de control de versiones, permitiendo registrar cada cambio realizado desde el inicio de la implementación hasta la versión final.

El repositorio utilizado fue:

https://github.com/23004004/cc8_proyect_phase_1

Git permitió:

- registrar el progreso del desarrollo;
- mantener un historial completo de modificaciones;
- documentar la evolución del proyecto mediante commits;
- facilitar la recuperación de versiones anteriores;
- respaldar el proceso de implementación.

El historial de commits constituye la evidencia cronológica del desarrollo realizado durante el proyecto.

14. Comunicación entre proyectos

Comunicación entre Implementaciones

El proyecto fue diseñado para que clientes y servidores desarrollados en diferentes lenguajes de programación pudieran comunicarse entre sí utilizando la especificación PRFC v3.

La comunicación se realiza mediante:

- UDP Broadcast para descubrir servidores.
- TCP para la comunicación del juego.
- Mensajes codificados según PRFC v3.
- Serialización y deserialización binaria.
- Compatibilidad entre diferentes implementaciones.

15. Uso de Inteligencia Artificial

Durante el desarrollo del proyecto se utilizó Codex - GPT-5.5 como herramienta de apoyo en las diferentes etapas del desarrollo. Su uso estuvo orientado a la planificación de la arquitectura, la adaptación del proyecto a los requisitos oficiales del curso y la revisión final del código.

La inteligencia artificial fue utilizada únicamente como apoyo técnico. Las decisiones de diseño, implementación, integración, pruebas y validación del funcionamiento fueron realizadas por los integrantes del equipo.

Prompt	Etapas	Objetivo
Prompt 1	Planificación inicial	Diseñar la arquitectura del proyecto y definir la estructura base antes de comenzar la implementación.
Prompt 2	Adaptación	Ajustar el proyecto a las instrucciones oficiales y migrar al protocolo PRFC v3.
Prompt 3	Revisión final	Refactorizar, mejorar la estructura del código y eliminar elementos sin uso sin modificar el comportamiento del proyecto.

Prompt 1: planificación y arquitectura inicial

Este prompt se utilizó al inicio del proyecto para diseñar la arquitectura antes de escribir código. En ese momento el proyecto aún se encontraba en fase de planificación y la comunicación estaba planteada inicialmente mediante JSON. Posteriormente, la implementación fue adaptada al protocolo oficial PRFC v3.

Necesito ayuda para un desarrollo. No quiero únicamente código, quiero arquitectura bien justificada, buenas prácticas y explicaciones técnicas.

Contexto del proyecto

Estoy desarrollando un proyecto universitario llamado Captura la Bandera.

Tecnologías obligatorias:

- Java 21.0.7 LTS
- IntelliJ IDEA
- Java Swing para la interfaz gráfica
- TCP Sockets
- Arquitectura Cliente-Servidor
- Comunicación mediante JSON
- UTF-8
- Proyecto Java simple, sin utilizar Maven o Gradle.

El objetivo principal **no** es la interfaz gráfica.

El objetivo principal es desarrollar un servidor robusto, escalable y compatible con implementaciones realizadas por otros compañeros en diferentes lenguajes de programación.

Debemos tomar en cuenta que nuestra implementación puede funcionar como servidor o como cliente, pero no como ambos al mismo tiempo.

Todos los clientes deberán poder conectarse entre sí independientemente del lenguaje utilizado.

Por esa razón, el protocolo de comunicación es el aspecto más importante.

También adjuntaré el documento oficial donde se especifica completamente el protocolo que todos los grupos deben respetar.

Ese documento tiene prioridad sobre cualquier decisión que propongamos.

Restricciones importantes

La interfaz gráfica será sencilla utilizando Swing.

No quiero utilizar motores gráficos.

La prioridad absoluta será:

1. Arquitectura.
2. Comunicación mediante sockets.
3. Sincronización del juego.
4. Concurrencia.
5. Compatibilidad con otros lenguajes.
6. Rendimiento.
7. Finalmente, la interfaz.

El servidor deberá soportar aproximadamente 50 jugadores simultáneos.

Debe minimizar problemas de latencia.

Debe evitar condiciones de carrera.

Debe tener un diseño limpio.

Debe ser fácilmente mantenible.

Debe seguir principios SOLID cuando sea posible.

El servidor será la única autoridad del juego.

Los clientes nunca decidirán posiciones ni resultados.

Toda la lógica debe ejecutarse en el servidor.

La interfaz únicamente representará el estado recibido.

Estructura inicial del proyecto

capture_the_flag/

```
└── src/
    ├── connect/
    │   ├── Client.java
    │   └── Server.java
    ├── model/
    │   └── Game.java
    ├── view/
    │   └── PanelGame.java
    └── Main.java
```

Actualmente estos archivos se encuentran prácticamente vacíos. Quiero que me ayudes a construir el proyecto paso a paso.

Forma de trabajar

No escribas todo el proyecto de una vez.

Trabajaremos de forma incremental.

Antes de escribir código:

- Analiza el problema.
- Identifica responsabilidades.
- Propón una arquitectura.
- Explica ventajas y desventajas.
- Después implementaremos solamente una pequeña parte.

Cuando propongas una solución:

- Explica por qué.
- Qué problema resuelve.
- Cómo afectará al resto del proyecto.

Siempre intenta que el código sea desacoplado.

Evita clases gigantes.

Evita duplicación.

Utiliza nombres claros.

Piensa como si este proyecto fuera software profesional.

Si detectas que la estructura puede mejorar, proponla antes de escribir código.

Cada decisión deberá considerar que posteriormente otros clientes escritos en Python, C#, Go, Node.js o C++ deberán poder conectarse sin modificar el protocolo.

No asumas información que no aparezca en el documento oficial del protocolo.

Si alguna regla entra en conflicto con una decisión de diseño, deberá respetarse el protocolo oficial.

Nuestro primer objetivo será diseñar correctamente la arquitectura del proyecto antes de escribir cualquier línea de código.

Prompt 2: adaptación a los cambios grupales y PRFC v3

Este prompt se utilizó cuando el proyecto ya tenía un avance inicial, pero surgieron nuevos requerimientos dentro del grupo y fue necesario adaptar la implementación al protocolo oficial PRFC v3 y a las instrucciones actualizadas del curso.

Hay que hacer varias mejoras, ya que como es grupal surgieron cambios y ya están las instrucciones; nosotros habíamos adelantado parte del desarrollo.

CapturaLaBandera.md fue la idea principal en la que se basó inicialmente el proyecto. Sin embargo, ahora debemos seguir la documentación oficial disponible en:

<https://github.com/erickm13/CC8-Protocolo/tree/main>

El archivo que nos interesa es PRFC-VERSION-3.

Ayúdame a realizar los cambios necesarios tomando en cuenta las instrucciones del PDF. El proyecto ya está implementado parcialmente, pero existen varios cambios relacionados con la interfaz gráfica y el protocolo.

Prompt 3: revisión y refactorización final

Este prompt se utilizó durante la etapa final del proyecto para mejorar la calidad del código sin modificar las partes que ya habían sido probadas con otros equipos.

Estamos en la etapa final del proyecto y quiero que lo revises con estos objetivos, en este orden:

1. Identifica código que no se usa (métodos, clases e imports muertos) y lístalo antes de eliminarlo, explicando brevemente por qué consideras que no se utiliza.
2. Señala puntos de mejora de legibilidad y estructura (nombres, organización de paquetes y clases, duplicación de código).
3. Refactoriza donde tenga sentido, sin cambiar el comportamiento funcional del proyecto.
4. Añade comentarios breves (una línea) únicamente en las partes clave o complejas, evitando párrafos largos.
5. No modifiques el protocolo, la lógica del servidor ni el funcionamiento del broadcast, ya que esas partes fueron probadas correctamente con otros compañeros.
6. Si es posible, mejora la estructura actual del proyecto tratando de no romper su funcionamiento.
7. Al finalizar, proporciona un resumen de todos los cambios realizados y verifica que el proyecto continúe compilando y funcionando correctamente.

Trabaja de forma incremental y avísame si algún cambio requiere mi confirmación antes de aplicarlo.